

第14章 类型系统进阶:结构类型、TS 与领域建模

建议学时:3-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解结构类型(鸭子类型)与 C 标称类型的本质差异
- 掌握运行时类型检查的三层防线:typeof / instanceof / shape 校验
- 掌握判别联合(tag 字段 + switch)的建模方法
- 掌握 JSON 作为"类型边界"的六种类型与四个转换坑
- 掌握深浅拷贝的边界:展开符、structuredClone 与手写递归
- 掌握空安全:?? 与 ?. 的组合规则
- 理解 TypeScript 与 JSDoc 渐进类型的方向,形成生产判断

💡 从 C 到 JavaScript:本章主题如何迁移

C 是标称类型(nominal typing): `struct Point` 与 `struct Circle` 就算布局一模一样也是两种类型,传参要显式强转;类型检查发生在编译期,运行期没有"类型"这个实体。JS 是结构类型(structural typing): 运行时只看对象的形状——"凡是长着 Point 样子的对象都能当 Point 用"。这把 C 里"编译期说了算"的权力移交给了运行时,代价是错误发现变晚,收益是极大的灵活性与可组合性(鸭子类型、动态 JSON、无模板的数据管道)。

类型不是"有或没有"的问题,而是 三道防线如何分工 :typeof 挡原始值,instanceof 挡类实例,手写 shape 校验挡"形状不符"的结构体。再加上判别联合(给对象一个 status 标签字段,用 switch 精确分支),C 程序员用 union + enum 才能表达的"一种数据多种形态",在 JS 里用普通对象就能优雅建模。

生产环境的真实图景是"渐进类型":核心边界(输入/输出/配置文件)要么用运行时校验兜底,要么引入 TypeScript 或 JSDoc 注解获得编译期检查;业务内部保持动态自由。第14章的实验(订单 + 商品 + 用户的领域模型)把本章所有技术串成一套真实系统:你会看到"校验函数 + 判别联合 + JSON 往返 + 深拷贝"如何组装出一个可运行、可维护的订单引擎——这是从"会写 C 程序"到"会设计 JS 系统"的最后一跃。

📦 前置知识自查

- C 的 struct/union/enum、强转、NULL 指针检查
- 第1章八种类型与 typeof;第6章原型链与 instanceof
- 第7章 JSON 序列化基础;第11章反射与 Object 反射全家

🚀 本章学习路线

1. **第一阶段(类型观)**:读 §14.1–§14.2,建立"结构类型 + 三层防线"的心智模型
2. **第二阶段(边界)**:读 §14.3–§14.5,吃透 JSON 转换坑、深浅拷贝与空安全
3. **第三阶段(工具)**:读 §14.6,理解 TS 与 JSDoc 渐进类型的定位
4. **第四阶段(建模)**:读 §14.7 并完成章末领域模型实验
5. **验收标准**:能用"校验函数 + 判别联合 + JSON 往返"独立建模一个带状态的业务实体

本章知识导图

- 第14章 类型系统进阶:结构类型、TS 与领域建模
 - §14.1 C 的标称类型 vs JS 的结构类型
 - §14.2 运行时类型检查的三层防线
 - §14.3 JSON:类型系统的外部边界
 - §14.4 深浅拷贝:边界与工具
 - §14.5 空安全:?? 与 ?. 的组合规则
 - §14.6 TypeScript 与 JSDoc:渐进类型的方向
 - §14.7 生产建议:运行时校验与领域建模

§14.1 C 的标称类型 vs JS 的结构类型

```
// C:标称类型 — 类型名即类型,布局相同也互不兼容
struct Point { int x, y; };
struct Circle { int x, y, r; }; // 布局包含 Point,但不是 Point

void draw(struct Point p) { /* ... */ }

struct Circle c = {1, 2, 5};
// draw(c); // 编译警告:不兼容的类型
draw((struct Point)c); // 强转:程序员负责,编译器闭嘴
```

```
// JS:结构类型 — 只看形状,不认名字
function draw(p) {
  // 不检查 p 是什么"类",只要求它有 x 和 y
  console.log('draw at (' + p.x + ', ' + p.y + ')');
}

draw({ x: 1, y: 2 }); // 普通对象,OK
draw({ x: 3, y: 4, r: 5 }); // 多一个 r,OK — 形状满足即可
draw({ x: 1 }); // 缺 y:运行时不报错,打印 (1, undefined)
```

结构类型的本质是 **"接口隐含于形状"**:调用方与被调方之间没有显式契约,契约就是"你需要哪些字段"。这带来两面的结果:

- **好的一面**:松耦合。测试时传一个假对象(测试替身)不需要继承任何类;函数天然多态——任何"长得像"的输入都能用。第6章的鸭子类型在这里有了理论依据。
- **代价**:拼错字段名不会报错,只会在运行时产生 undefined 与神秘 bug。C 编译器替你把关,现在要自己把——这就是 §14.2 三层防线存在的理由。

⚠ 常见误区 14.1:字段名拼错是"小事"

`user.name` 拼成 `user.nane`:不报错,拿到 undefined,后续拼接字符串变成 "undefined",存进数据库才发现。C 里这是编译错误,JS 里是"迟到的数据事故"。对策:关键字段在入口处做 shape 校验(§14.2),或引入 TS/JSDoc(§14.6)。

§14.2 运行时类型检查的三层防线

三层防线各管一段,生产代码按"成本从低到高"使用:

```
// 第一层:typeof — 管原始值(成本最低)
function toStr(x) {
  if (typeof x === 'string') return x;           // 已经是字符串
  if (typeof x === 'number') return String(x);   // 数字转字符串
  return '';                                     // 其余给空串
}

// 第二层:instanceof — 管类实例
class Account { /* ... */ }
function deposit(a, amount) {
  if (!(a instanceof Account)) throw new TypeError('需要 Account 实例');
  // 跨 realm 场景(第11章)注意 instanceof 会失效,用品牌字段替代
}

// 第三层:shape 校验 — 管"结构体形状"(成本最高,用在边界)
function isProduct(obj) {
  return (
    obj !== null &&
    typeof obj === 'object' &&
    typeof obj.id === 'string' &&           // 字段存在且类型正确
    typeof obj.name === 'string' &&
    typeof obj.price === 'number' &&
    Number.isFinite(obj.price) &&           // 拒绝 NaN/Infinity
    obj.price >= 0                           // 值域约束
  );
}
const raw = JSON.parse(await fetch('/api/products/1').then((r) => r.json()));
if (!isProduct(raw)) throw new Error('商品数据格式错误: ' + JSON.stringify(raw));
```

判别联合:当同一份数据有多种形态时(C 的 union + enum 组合),用一个"标签字段"区分:

```
// 订单三态:tag 字段 status + switch 精确分支
function formatOrder(order) {
  switch (order.status) {
    case 'pending': return '待支付: ' + order.id;
    case 'paid':     return '已支付: ' + order.id + ', 金额 ¥' + order.total;
    case 'cancelled': return '已取消: ' + order.id + ' (原因: ' + order.reason + ')';
    default:
      // 未知分支必须抛错:宁可让系统亮红灯,也不要静默返回错误数据
      throw new Error('未知订单状态: ' + order.status);
  }
}
```

判别联合三原则: 标签字段放显眼位置(status)、default 分支必抛错、每个分支只访问自己该有的字段(cancelled 才有 reason)。这正是 C 里 enum + union + switch 的运行时版,且更安全——没有"谁都能乱读 union"的问题。

§14.3 JSON:类型系统的外部边界

JSON 只有六种类型:null、boolean、number、string、array、object。JS 的世界比它宽,跨边界时会发生隐式转换:

```
const order = {
  id: 1,
  createdAt: new Date(),      // Date 不在 JSON 类型里
  note: undefined,           // undefined 没有 JSON 表示
  score: NaN,                 // NaN 不是 JSON number
  big: 9007199254740993n,     // BigInt 不支持序列化
};

const json = JSON.stringify(order);
console.log(json);
// {"id":1,"createdAt":"2026-08-26T08:00:00.000Z","score":null}
// Date → ISO 字符串;undefined 属性整个被丢弃;NaN → null;BigInt 直接抛错
```

- **Date → 字符串**:序列化后是 ISO 字符串,反序列化回来仍是字符串,不是 Date。JSON 往返改变类型——领域模型里要显式转换(`new Date(obj.createdAt)`)。
- **undefined/NaN**:undefined 属性被丢弃,NaN/Infinity 变成 null。数据静默变形,是“看起来没丢,用起来错了”的经典来源。
- **大整数**:超过 $2^{53} - 1$ 的整数 id(如雪花 ID)精度丢失。生产规则:大 ID 用字符串承载。
- **循环引用**:JSON.stringify 抛 TypeError。深度遍历要先做防环(第11章的 Set 技巧)。

⚠ 常见误区 14.2:JSON 是“无损的”

JSON 只是序列化格式,不是 JS 的镜像。Date/undefined/NaN/BigInt 四个类型跨边界都会变形或报错。规则:跨 JSON 边界前,先把类型显式转换成 JSON 六型之一(日期 → ISO 字符串、大整数 → 字符串、undefined → 省略),反序列化后再显式还原。

§14.4 深浅拷贝:边界与工具

```
const a = { user: { name: '李雷', tags: ['a', 'b'] } };

// 浅拷贝:{...a} 只复制第一层,内层对象仍然共享
const shallow = { ...a };
shallow.user.name = '韩梅梅';
console.log(a.user.name);      // '韩梅梅' ← 原对象被污染了!

// 深拷贝:structuredClone(ES2022 内置,支持 Date/Map/Set/循环引用)
const deep = structuredClone(a);
deep.user.name = '张三';
console.log(a.user.name);      // '韩梅梅' — 原对象不受影响

// 浅拷贝的另一写法(老代码常见)
const old = Object.assign({}, a);
```

深浅拷贝的边界要记牢:

- **{...obj} / Object.assign**:只复制可枚举自身属性,且只复制“引用”本身——内层对象/数组仍是同一份。90% 的“拷贝 bug”都是把它当深拷贝用。
- **structuredClone**:现代深拷贝首选:支持循环引用、Date、Map、Set、TypedArray。限制:函数与类实例(非结构化数据)会抛 DataCloneError,Symbol 键会丢失。

- **手写递归 clone:**教学与特殊场景(过滤字段、转换类型)才需要;必须带 WeakMap 防循环引用:

```
function deepClone(obj, seen = new WeakMap()) {
  if (obj === null || typeof obj !== 'object') return obj; // 原始值原样返回
  if (seen.has(obj)) return seen.get(obj); // 防循环引用
  const clone = Array.isArray(obj) ? [] : {};
  seen.set(obj, clone);
  for (const [k, v] of Object.entries(obj)) {
    clone[k] = deepClone(v, seen); // 递归
  }
  return clone;
}
```

⚠ 常见误区 14.3:改"拷贝"影响了原数据

排查顺序:先确认是不是浅拷贝共享了内层引用。生产规则:默认用 structuredClone;只在"确信内层会被替换而不会被修改"时用浅拷贝(如 React 的不可更新惯例)。

§14.5 空安全:?? 与 ?. 的组合规则

```
// C:NULL 指针解引用 → 段错误,程序当场崩溃
// JS:null/undefined 上访问属性 → TypeError(同理,但可防御)

// 空值合并 ?? :只在 null/undefined 时取默认值
const port = process.env.PORT ?? 8080;
const name = user.name ?? '匿名';
const zero = 0 ?? 100; // 0 — 空值合并不吞 0
const empty = '' ?? '默认'; // '' — 空值合并不吞空串

// 对比 || :把 falsy 值一视同仁(0/''/NaN 都会被替换)
const a = 0 || 100; // 100 ← 逻辑或吞掉了 0
const b = 0 ?? 100; // 0 ← 空值合并保留 0

// 可选链 ?. :路径上任何一环为空,整个表达式短路为 undefined
const city = user?.address?.city ?? '未知';
// 等价于:
// const city = (user != null && user.address != null)
//   ? (user.address.city ?? '未知')
//   : '未知';

// 组合调用:函数可能不存在时
const result = config?.parser?.call(config, raw) ?? null;
```

空安全的黄金组合:?. 负责"路",?? 负责"默认值",两者总是成对出现。对照 C:JS 的空访问不会崩溃进程,但会静默传播 undefined——所以"默认值"必须显式给出,否则 undefined 会一路流到 UI 层变成 "undefined" 字样。

表达式	x = 0	x = ''	x = null	x = undefined
x '默认'	'默认'	'默认'	'默认'	'默认'
x ?? '默认'	0	''	'默认'	'默认'

§14.6 TypeScript 与 JSDoc:渐进类型的方向

```
// TypeScript — 带类型的 JS 超集:编译期检查,编译产物就是 JS
interface Product {
  id: string;
  price: number;
}

function total(products: Product[]): number {
  return products.reduce((s, p) => s + p.price, 0);
}

total([{ id: 'a', price: 9.9 }]); // 编译通过
// total([{ id: 'a' }]); // 编译错误:缺少 price — 错误在写代码时发现
```

TS 对 C 程序员非常友好:它把"标称感"带回来了——interface 声明形状,函数签名约束参数,编辑器即时报错,像 C 的编译期检查提前到输入阶段。但它不改变运行行为:类型在编译时被擦除,运行时仍是动态 JS——所以 TS 检查通过不意味着数据合法(网络 JSON 依然要校验)。

联合类型是 TS 对判别联合的静态化:

```
type OrderStatus = 'pending' | 'paid' | 'cancelled';

interface Order {
  id: string;
  status: OrderStatus;
  total: number;
}

// status 写错成 'penging' 会在编译期被拦下 — 这就是 §14.2 判别联合的静态版
```

JSDoc 渐进类型:不想引入构建步骤时,用 JSDoc 注释获得同样的编辑器检查(零依赖,VS Code 原生支持):

```
// @ts-check ← 文件顶部的注释:开启本文件类型检查
/** @param {{ id: string, price: number }[]} products */
function total(products) {
  return products.reduce((s, p) => s + p.price, 0);
}

/** @typedef {{ id: string, price: number }} Product */
/** @type {Product[]} */
const list = [];
```

⚠ 常见误区 14.4:TS 检查通过 = 数据安全

TS 的类型在运行时被擦除,JSON.parse 出来的数据它管不着。生产规则:TS/JSDoc 管"代码内部",运行时校验管"系统边界"——网络响应、文件内容、用户输入,一律在入口用 shape 校验(§14.2)把关,再信任内部。

§14.7 生产建议:运行时校验与领域建模

把全章收敛成三条工程原则:

- **原则一(边界校验):**在"不可信数据"进入系统的四个入口做校验——HTTP 请求体、JSON 文件、环境变量/配置、用户输入。校验失败立即抛错,错误信息要带原始数据片段,方便排查。
- **原则二(判别联合优先):**一个实体有多种状态/形态时,用 status 标签 + switch,default 抛错。不要用"一堆可选字段 + 到处 if"——那会让非法状态静默存在。
- **原则三(默认值显式):**所有"可能为空"的读取都用 ?? 给出显式默认;绝不把 undefined 留在数据里流向 UI 或数据库。C 程序员把它想成"每次解引用前先判 NULL"的纪律,只是更优雅。

 **常见误区 14.5:校验函数到处复制粘贴**

isProduct 这种 shape 校验是纯函数、无状态——把它放进专门的 domain/validators 文件,统一导出;校验失败抛出的错误统一包装成带字段名的 Error,方便上层日志聚合。生产包(如 zod)提供声明式 schema,一行顶十行,值得了解。

本章速查表

主题	速查要点
标称类型	C:类型名即类型;布局相同也不兼容;强转是程序员承诺
结构类型	JS:运行时只看形状;缺字段不报错,产生 undefined
三层防线	typeof 管原始值;instanceof 管类实例;shape 校验管形状与值域
判别联合	status 标签 + switch;default 必须抛错;分支只碰自己的字段
JSON 六型	null/boolean/number/string/array/object;其余类型跨界会变形
JSON 转换坑	Date→字符串;undefined 属性丢弃;NaN→null;BigInt 抛错
大整数	超过 2^53-1 精度丢失;生产用字符串承载大 ID
浅拷贝	{...a}/Object.assign:只复制第一层,内层共享引用
深拷贝	structuredClone:循环引用/Date/Map/Set 安全;函数不可克隆
手写 clone	递归 + WeakMap 防环;只在特殊场景(过滤/转换)需要
?? 空值合并	只在 null/undefined 时取默认;0 与 "" 保留
? 可选链	路径上任一环为空即短路;与 ?? 成对使用
TypeScript	编译期检查、编译后擦除;interface/联合类型;不替代运行时校验
JSDoc 渐进	// @ts-check + /** @type */ 零依赖获得编辑器检查
生产三原则	边界校验 / 判别联合优先 / 默认值显式

最小必会清单

- 能解释标称类型与结构类型的差异及各自的代价
- 能说出三层防线的分工与使用顺序
- 能写出 isProduct 风格的 shape 校验函数(含值域检查)
- 能用 status + switch 建模多态实体,并坚持 default 抛错

- 能说出 JSON 六种类型与四个转换坑(Date/undefined/NaN/BigInt)
- 能说出 {...a} 与 structuredClone 的边界,并给出排查思路
- 能写出带 WeakMap 防环的手写深拷贝
- 能解释 ?? 与 || 的区别,并正确使用 ?. 与 ?? 组合
- 能说出 TS 与 JSDoc 的定位(编译期/编辑器检查,运行期擦除)
- 能按"边界校验、判别联合、显式默认值"三原则设计一个领域模型

自测题

Q 1. 结构类型与标称类型的本质区别是什么?JS 为此付出的代价是什么?

✓ 参考答案

标称类型看"名字/声明",布局相同也互不兼容;结构类型只看形状,有需要的字段就能用。代价:拼错字段不报编译错,运行时产生 undefined;错误发现变晚,需要自行加强校验。

Q 2. {...obj} 是深拷贝吗?请举一个它"污染原对象"的具体例子。

✓ 参考答案

不是,只复制第一层。例:const b = { ...a }; b.user.name = 'x' 会修改 a.user.name,因为内层 user 是共享引用。要深拷贝用 structuredClone。

Q 3. 写出 0 ?? 5 与 0 || 5 的结果,并解释差异。

✓ 参考答案

0 ?? 5 是 0(空值合并只在 null/undefined 时取默认,0 是合法值被保留);0 || 5 是 5(逻辑或把 0 当 falsy 替换)。配置默认值场景优先用 ??。

Q 4. JSON.stringify({ d: new Date(), u: undefined, n: NaN }) 输出什么?

✓ 参考答案

{"d":"2026-08-26T...(ISO 字符串)","n":null}:Date 变字符串、undefined 属性被丢弃、NaN 变 null。这就是"JSON 往返改变类型"的三个例子。

Q 5. structuredClone 不能克隆什么?遇到循环引用会怎样?

✓ 参考答案

不能克隆函数与类实例(抛 DataCloneError),Symbol 键丢失;循环引用是支持的(内置处理),不需要手动防环——这正是它相对 JSON.stringify 的优势。

Q 6. 判别联合的 default 分支为什么必须抛错而不是返回一个兜底值?

✅ 参考答案

返回兜底值会掩盖非法状态——数据从“脏”变成“静默的错误结果”,后续在业务深处爆炸,极难定位。抛错让错误在边界处立刻暴露,带着上下文日志,符合“失败要早”原则。

章末实验题:订单领域模型

实验 14:订单引擎(商品 + 用户 + 订单领域模型)

实验目标:用“校验函数 + 判别联合 + JSON 往返 + 深拷贝”建模一个可运行的订单系统,覆盖全章知识点。

实验要求:

- 任务 1(覆盖:shape 校验):createProduct/createUser/createOrder 三个工厂,入参校验失败抛带字段名的 Error
- 任务 2(覆盖:判别联合):订单状态 pending/paid/cancelled 用 status 标签建模;describe(order) 按状态输出不同描述,default 抛错
- 任务 3(覆盖:金额与精度):金额用“整数分”存储,展示 priceInCents 计算,避免浮点误差;大 ID 用字符串
- 任务 4(覆盖:JSON 往返):订单序列化保存、反序列化还原;Date 字段显式转换;undefined 属性显式省略
- 任务 5(覆盖:深拷贝):用 structuredClone 复制订单并修改副本,验证原订单不受影响
- 任务 6(覆盖:空安全):读取可能缺失的用户地址,用 ?. + ?? 输出“未知城市”
- 任务 7(覆盖:JSDoc):为每个工厂函数加 /** @param */ 与 /** @returns */ 注解(用 // @ts-check 验证编辑器提示)

实验环境:Node.js 20+,运行 `node order.js`。

第一步 建校验:把 isProduct/isUser 写成独立纯函数,工厂复用它们——边界校验与业务逻辑分离

第二步 建模:订单 = 用户 + 商品数组 + 状态机;金额一律用分(整数),输出时再转元

第三步 后串流程:创建订单 → 支付 → 序列化 → 反序列化 → 深拷贝实验,输出完整演示日志

✅ 参考答案要点

核心:纯校验函数 + status 判别联合 + 分单位金额 + JSON 往返 + structuredClone 深拷贝。约 130 行,覆盖全章知识点。

```

// order.js — Node.js 20+ 运行:node order.js
// @ts-check
'use strict';

// ===== 校验层:纯函数,无副作用,可复用 =====
/** @param {unknown} v @returns {v is { id: string, name: string, priceCents: number }} */
function isProduct(v) {
  return (
    v !== null && typeof v === 'object' &&
    typeof v.id === 'string' && v.id.length > 0 &&
    typeof v.name === 'string' &&
    typeof v.priceCents === 'number' &&
    Number.isInteger(v.priceCents) && v.priceCents >= 0
  );
}

/** @param {unknown} v @returns {v is { id: string, name: string, city: string }} */
function isUser(v) {
  return (
    v !== null && typeof v === 'object' &&
    typeof v.id === 'string' &&
    typeof v.name === 'string' &&
    (v.city === undefined || typeof v.city === 'string')
  );
}

// ===== 工厂层:入参校验,失败立即抛错 =====
/** @param {string} id @param {string} name @param {number} priceCents */
function createProduct(id, name, priceCents) {
  const p = { id, name, priceCents };
  if (!isProduct(p)) throw new Error('商品参数非法: ' + JSON.stringify(p));
  return p;
}

/** @param {string} id @param {string} name @param {string} [city] */
function createUser(id, name, city) {
  const u = { id, name, city };
  if (!isUser(u)) throw new Error('用户参数非法: ' + JSON.stringify(u));
  return u;
}

/** @param {{ id: string, user: object, items: object[] }} args */
function createOrder(args) {
  if (!isUser(args.user)) throw new Error('订单用户非法');
  if (!Array.isArray(args.items) || args.items.some((it) => !isProduct(it))) {
    throw new Error('订单商品列表非法');
  }
  const totalCents = args.items.reduce((s, it) => s + it.priceCents, 0);
  return {
    id: args.id,
    status: 'pending', // 判别联合:初始状态唯一合法值
    user: args.user,
    items: args.items,
    totalCents,
    createdAt: new Date(),
  };
}

```

```

    paidAt: null,                // 显式 null:表示"尚不存在",而非 undefined
  };
}

// ===== 状态机:判别联合的 switch,default 必须抛错 =====
function pay(order) {
  if (order.status !== 'pending') throw new Error('只有 pending 可支付: ' + order.status);
  order.status = 'paid';
  order.paidAt = new Date();
}
function cancel(order, reason) {
  if (order.status === 'paid') throw new Error('已支付订单不能取消');
  order.status = 'cancelled';
  order.reason = reason;
}

function describe(order) {
  const yuan = (order.totalCents / 100).toFixed(2);
  switch (order.status) {
    case 'pending': return '待支付 ' + order.id + ', 金额 ¥' + yuan;
    case 'paid':    return '已支付 ' + order.id + ', 金额 ¥' + yuan;
    case 'cancelled': return '已取消 ' + order.id + ': ' + order.reason;
    default: throw new Error('未知订单状态: ' + order.status);
  }
}

// ===== JSON 往返:日期显式转换,undefined 显式省略 =====
function toJSON(order) {
  return {
    id: order.id,
    status: order.status,
    user: order.user,
    items: order.items,
    totalCents: order.totalCents,
    createdAt: order.createdAt.toISOString(), // Date → 字符串
    paidAt: order.paidAt ? order.paidAt.toISOString() : null,
  };
}
function fromJSON(json) {
  const order = createOrder({
    id: json.id,
    user: json.user,
    items: json.items,
  });
  order.status = json.status; // 还原状态
  order.createdAt = new Date(json.createdAt); // 字符串 → Date
  order.paidAt = json.paidAt ? new Date(json.paidAt) : null;
  order.totalCents = json.totalCents;
  return order;
}

// ===== 主流程 =====
function main() {
  const apple = createProduct('P-001', '苹果', 550); // ¥5.50,分单位
  const milk = createProduct('P-002', '牛奶', 1200); // ¥12.00

```

```

const user = createUser('U-9001', '李雷', '北京');

const order = createOrder({ id: 'ORD-9001', user, items: [apple, milk] });
console.log(describe(order));           // 待支付 ORD-9001, 金额 ¥17.50
pay(order);
console.log(describe(order));           // 已支付 ...
cancel(order, '用户取消');              // 抛错:已支付不能取消

// 深拷贝:改副本不影响原件
const copy = structuredClone(order);
copy.user.name = '韩梅梅';
console.log('原件用户:', order.user.name); // 李雷
console.log('副本用户:', copy.user.name);   // 韩梅梅

// JSON 往返
const json = JSON.stringify(toJSON(order));
const restored = fromJSON(JSON.parse(json));
console.log('往返一致:', restored.id === order.id &&
  restored.totalCents === order.totalCents);
console.log('还原的日期类型:', restored.createdAt instanceof Date); // true

// 空安全:地址可能缺失
const guest = createUser('U-9002', '韩梅梅'); // 没有 city
console.log('城市:', guest?.city ?? '未知');    // 未知
}
main();

```

设计说明:金额用"分"整数存储,totalCents 由 items 归约计算,输出时再转元——规避浮点误差;paidAt 用显式 null 而非 undefined,让 JSON 往返行为可预测;createOrder 在构造时即校验,保证系统内不存在非法订单;copy 演示 structuredClone 深拷贝隔离;guest?.city ?? '未知' 是空安全黄金组合。运行后应看到:支付成功描述、取消抛错、深拷贝互不影响、往返一致为 true、日期还原为 Date 实例。

全书写到这里。从第1章的数字与类型,到第14章的领域建模,你已经走完了"从 C 到 JavaScript"的完整进阶之路。祝你在 Node.js 与浏览器两个世界都写出清晰、健壮、可维护的代码。